

Nota sobre el servicio de archivos de medios en Django

1. Arquitectura de despliegue

En un entorno de despliegue ideal, una aplicación basada en Django distribuye sus responsabilidades entre distintos servicios. Un servidor como Gunicorn (o similar) se encarga de servir la aplicación y gestionar las vistas. Herramientas como Whitenoise gestionan los archivos estáticos, como hojas de estilo y módulos JavaScript. Finalmente, servidores como Nginx o Apache se ocupan de servir contenido multimedia, como imágenes, audio o vídeo.

Estos últimos son especialmente eficientes manejando datos binarios y permiten utilizar sistemas de almacenamiento más económicos, lo que mejora el rendimiento y la escalabilidad del sistema.

2. Problema detectado en la práctica

Durante el desarrollo de la Práctica 4 (P4), se ha detectado un problema en el despliegue en Render. La aplicación únicamente sirve archivos de medios cuando la variable `DEBUG=True`, pero deja de hacerlo cuando `DEBUG=False`.

Este comportamiento es coherente con el funcionamiento por defecto de Django. En modo desarrollo, el framework sirve automáticamente los archivos de medios para facilitar las pruebas. Sin embargo, en entornos de producción, deja de hacerlo y delega esta responsabilidad en servidores web especializados.

3. Solución aplicada

Para resolver este problema en el entorno actual, limitado por el uso de una cuenta gratuita en Render, se puede modificar el archivo `urls.py` para permitir que Django sirva los archivos de medios manualmente.

La modificación es la siguiente:

```
from django.contrib import admin
from django.urls import path, include, re_path
from django.views.static import serve
from django.conf import settings
from django.conf.urls.static import static

urlpatterns = [
    path('admin/', admin.site.urls),
    path('api/v1/', include('djoser.urls')),
    path('api/v1/', include('djoser.urls.authtoken')),
    path('api/v1/', include('api.urls')),
    re_path(r'^media/(?P<path>.*)$', serve,
            {'document_root': settings.MEDIA_ROOT}),
]
```

La línea añadida es la que utiliza `re_path`, que permite a Django servir directamente los archivos ubicados en `MEDIA_ROOT`.

4. Consideraciones

En condiciones normales, Django no está diseñado para servir archivos de medios en entornos de producción. Cuando la variable `DEBUG=True`, el propio framework los sirve automáticamente para facilitar el desarrollo. Sin embargo, al establecer `DEBUG=False`, Django deja de hacerlo, esperando que esta responsabilidad recaiga en un servidor web especializado como Nginx o Apache.

La solución propuesta constituye un *workaround* funcional en el contexto de esta práctica y dadas las limitaciones del despliegue en Render. No obstante, es importante señalar que no se trata de una práctica recomendada para entornos de producción reales, ya que puede afectar negativamente al rendimiento y la escalabilidad de la aplicación.